Report Transformer Models

Project by Jakob Berg & Tobias Munch

Division of tasks:

We collaborated on both tasks throughout the project, approach, hyperparameter choices and debugging were discussed jointly. The bullets below describe who took the lead on each component; everything else was done together or by the other person.

Part 1 — Encoder-only IMDb sentiment classifier

- Tobias: Encoder implementations.
- Jakob: pre-processing and tokenization, `IMDBDataset` / `create_mask`, positional encoding, training and evaluation loops, `classify_review`.

Part 2 — Decoder-only GooAQ chatbot

- Tobias: decoder block, BPE tokenizer, and the inference / sampling code.
- Jakob: data preparation and dataset class, model assembly, training run on HubroHub.

Disclosure of implementation:

When working together on this project, we mostly worked alongside each other in pairs, with some work being done separately. The implementation is built on top of PyTorch. In **Part 1**, the core transformer components (multi-head self-attention, encoder block, sinusoidal positional encoding and the classification head) were written from scratch using only `torch.nn.Linear`, `torch.nn.LayerNorm`, `torch.nn.Embedding` and `torch.nn.Dropout`. In **Part 2**, the `DecoderBlock` in `model.py` is assembled from the same primitives, but the masked multi-head attention inside it uses the built-in `torch.nn.MultiheadAttention` (with a causal mask and a key-padding mask) as the project description asks. For tokenization we used Hugging Face's `tokenizers` library: a word-level tokenizer in Part 1 and a byte-pair-encoding tokenizer in Part 2; `datasets` was used to load the IMDb and GooAQ corpora. Supporting libraries were `numpy` for array operations, `matplotlib` for plots (token-length histograms, loss curves) and `tqdm` for training progress bars.

Training in Part 1 was run on UiB's HubroHub computer; Part 2's training was also run on HubroHub since the larger model and longer sequences made CPU training impractical.

Introduction

In this project, we implement and train two transformer models from scratch and use them to solve two different NLP tasks: sentiment classification and free-form text generation. Both models are built from the same core building blocks described in *Attention is All You Need* (Vaswani et al., 2017), embedding, sinusoidal positional encoding, multi-head self-attention, position-wise feed-forward networks, layer normalization and residual connections, but they differ in the direction of attention and in the output head.

In the **sentiment classification task (Part 1)**, we build an **encoder-only** transformer that reads a movie review from the IMDb dataset and predicts whether its sentiment is *positive* or *negative*. A `[CLS]` token is prepended to each sequence, passed through a stack of encoder blocks with bidirectional self-attention, and

its final embedding is fed into a linear classification head. The model is trained with binary cross-entropy on 25 000 reviews, with a padding mask preventing attention to `[PAD]` tokens.

In the **text generation task (Part 2)**, we build a **decoder-only** transformer (GPT-style) that is trained on a subset of the GooAQ question/answer corpus. Because the model is autoregressive, each block uses a *causal* attention mask that prevents each position from attending to future tokens, in addition to the padding mask. Tokenization moves from word-level to byte-pair encoding so the model can handle rare and unseen words by composing them from sub-word units. After training, we sample answers from the model and compare different decoding strategies (greedy vs. temperature/top-k sampling) to see how they affect fluency, diversity and hallucination.

Part 1: Encoder-only Model for Movie Review Sentiment Classification

1. Pre-processing and Tokenization of the IMDb Reviews

The IMDb dataset contains 50,000 movie reviews labelled as either *positive* or *negative*. We used 25,000 reviews for training, 5,000 for validation, and 20,000 for testing (the validation split was taken from the original test set using a fixed seed).

Pre-processing. Each review is cleaned in three steps:

1. HTML tags such as `<br />` are stripped with a regular expression.
2. Special characters are removed, keeping only letters, digits, spaces and the punctuation `,.!?`.
3. The text is lowercased.

This removes noise introduced by the HTML-formatted source while preserving punctuation that may carry sentiment information.

Tokenization. We use a **word-level tokenizer** from the `tokenizers` library with a `Whitespace` pre-tokenizer. The tokenizer is trained on the (pre-processed) training set with three special tokens:

- `[PAD]` – used for padding shorter sequences to the same length.
- `[UNK]` – used for out-of-vocabulary words.
- `[CLS]` – prepended to every sequence; its final embedding is used as the sentence representation for classification.

To keep the vocabulary small we set `min_frequency=10` and `vocab_size=10 000`, so rare words get mapped to `[UNK]`.

Dataset class. In `IMDBDataset.__getitem__` each review is encoded, truncated to `max_length - 1` tokens, prepended with the `[CLS]` id and padded up to `max_length = 256` with `[PAD]`. The method returns a tensor of token ids and the integer label. A histogram of token lengths on the training set showed that 256 tokens covers the majority of reviews while keeping the model tractable.

2. The Encoder-only Transformer and Attention

The model is built from the building blocks described in *Attention is All You Need* (Vaswani et al., 2017). Its forward path is:

1. **Embedding layer** – maps token ids to vectors of dimension `embedding_dim = 96`.
2. **Sinusoidal positional encoding** – pre-computed once using `pe[:, 2i] = sin(pos / 10000^(2i/d))` and `pe[:, 2i+1] = cos(pos / 10000^(2i/d))`, and added to the embeddings so the model knows token positions.
3. **Stack of `num_layers = 3` Encoder Blocks**, each consisting of:
   - `LayerNorm` → `MultiHeadAttention` (with padding mask) → Dropout → residual
   - `LayerNorm` → MLP (`Linear(d, 4d)` → GELU → Dropout → `Linear(4d, d)` → Dropout) → residual
4. **Classification head** – we take the final embedding of the `[CLS]` token (position 0), pass it through a linear layer with a single output, and apply sigmoid to get a probability in [0, 1].

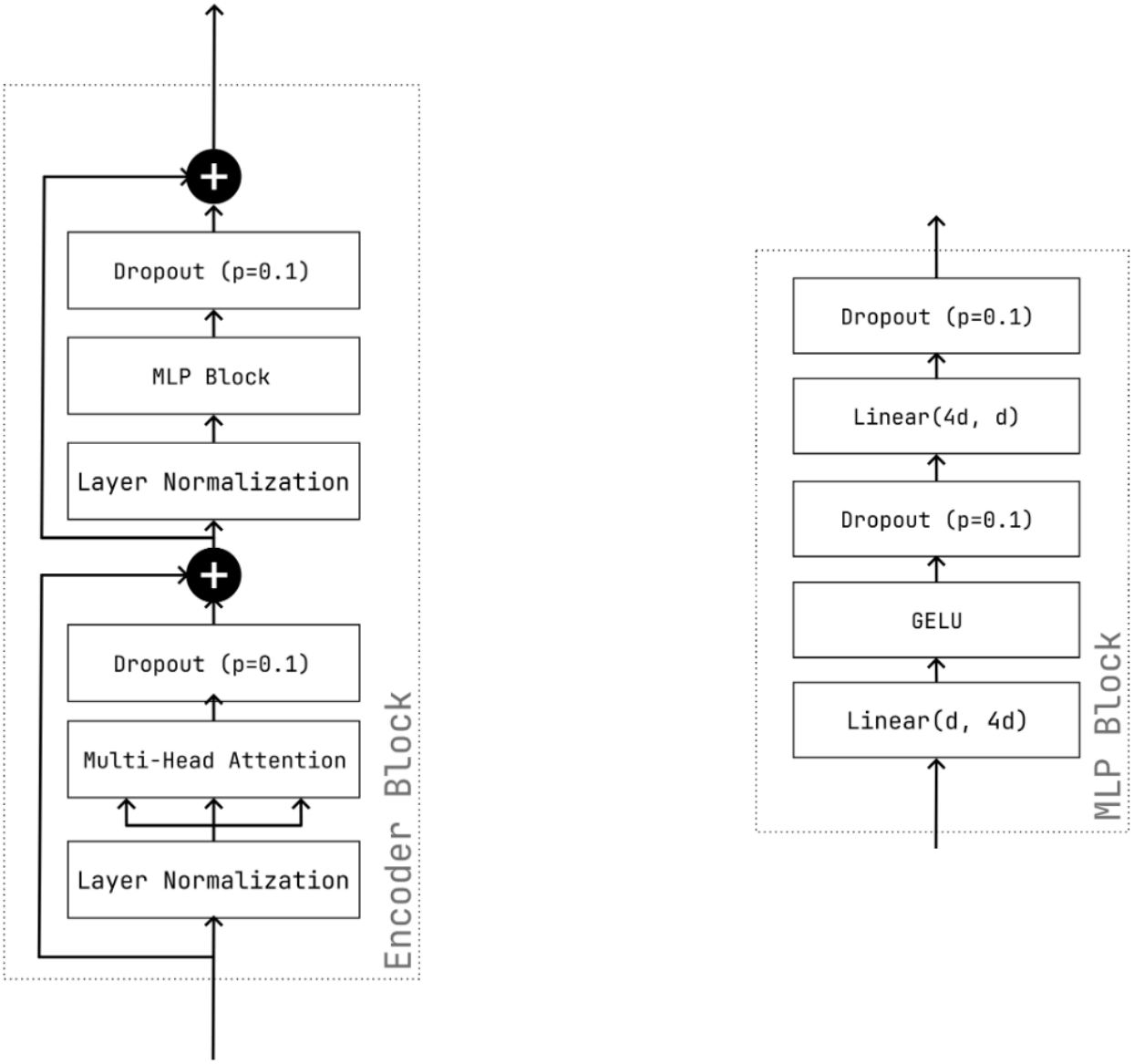

Figure 1: One encoder block. The input is first normalized and passed through multi-head self-attention followed by dropout before the residual connection, then normalized again and passed through the position-wise MLP (`Linear → GELU → Dropout → Linear → Dropout`) before a second residual connection. Dropout is applied after each sub-layer output (and inside the MLP after the activation) to regularize training; at inference it is a no-op. We stack three of these blocks before the `[CLS]` token is read off and fed to the classification head.

Scaled dot-product attention. For queries `Q`, keys `K` and values `V`,

`Attention(Q, K, V) = softmax(QKᵀ / √d_k) V`.

We compute `Q`, `K`, `V` by three linear projections of the input, reshape them to `(batch, num_heads, seq_len, head_dim)` so that all `num_heads = 4` heads are computed in parallel, and then concatenate the heads before a final linear projection. The padding mask (a boolean tensor with `True` at padding positions) is broadcast to the attention scores and used with `masked_fill(..., -inf)` so padding positions receive zero weight after softmax.

3. Approach to the Tasks

- **Task 1 (`IMDBDataset.__getitem__`).** Encode, truncate to `max_length - 1`, prepend `[CLS]`, right-pad with `[PAD]`.
- **Task 2 (`create_mask`).** Return `sequences == pad_id` as a boolean tensor of shape `(batch, seq_len)`.
- **Task 3 (`MultiheadAttention`).** Implemented from scratch using only one `nn.Linear` for each of Q, K, V and one for the output projection. Heads are handled with `view` + `transpose` rather than separate per-head layers, which is faster.
- **Task 4 (`EncoderBlock`).** Pre-norm variant: `x + Dropout(Attn(Norm(x)))` then `x + Dropout(MLP(Norm(x)))`, matching Figure 1 in this report.
- **Task 5 (`PositionalEncoding`).** Pre-computed in `__init__` with vectorized `torch.arange` / `torch.sin` / `torch.cos`, stored via `register_buffer` (so it moves with the model to the correct device), and added to the input in `forward`.
- **Task 6/7 (Training & Evaluation).** Trained for 3 epochs with AdamW (`lr = 1e-3`, `weight_decay = 1e-3`), `BCELoss`, gradient clipping at 10.0, batch size 64. The total model has **1,295,617 parameters**.
- **Task 8 (Custom reviews).** Wrote `classify_review` which preprocesses, tokenizes and pads the review in the same way as the training pipeline and then returns the predicted sentiment and probability.

4. Results

Epoch	Train Loss	Train Acc.	Val Loss	Val Acc.
1	0.5351	0.7120	0.4129	0.8106
2	0.3724	0.8331	0.3689	0.8368
3	0.3023	0.8726	0.3855	0.8406

Test accuracy: 0.8358 on the held-out 20k reviews.

Training loss decreases monotonically across the three epochs, while validation accuracy keeps improving and lands inside the 80–85% range suggested in the project description. Validation loss, however, drops in epoch 2 and then ticks up slightly in epoch 3 (0.3689 → 0.3855) even as validation accuracy keeps rising, an early sign that the model is starting to become over-confident on some examples. The gap between training accuracy (87.3%) and validation accuracy (84.1%) in the final epoch is still moderate (~3 pp), so the over-fitting is mild after three epochs.

5. Discussion and Challenges

- The most subtle part was the **multi-head attention**. Getting the `view`/`transpose` order right so that the `num_heads` axis ends up in position 1 (and the `seq_len` axis in position 2) is easy to get wrong; a

wrong reshape silently trains but collapses heads. Writing out the shapes on paper before coding helped.
- The **padding mask** also needs shape `(batch, 1, 1, seq_len)` when applied to scores of shape `(batch, heads, seq_len, seq_len)`. The broadcasting hides mistakes because PyTorch will happily broadcast wrong shapes silently if the sizes happen to line up.
- **Pre-norm vs. post-norm** in the encoder block: we used pre-norm (LayerNorm *before* attention / MLP), which tends to train more stably on small models than the original post-norm version.
- Training was run on UiB's HubroHub computer. Each epoch took about 16 seconds thanks to the small model and short max sequence length.
- The model still misclassifies reviews containing heavy sarcasm or mixed sentiment. A word-level tokenizer with `[UNK]` for rare words also means that ironic phrasing relying on uncommon words is partially erased before the model even sees it.

6. Custom Review Predictions

Three real reviews were picked from IMDb to stress-test the model. The rating next to each movie is the score the original reviewer gave the film out of 10.

#	Movie (reviewer rating)	Review (shortened)	My sentiment	Model prediction	Probability(positive)
1	Sharknado (4/10)	*"Sharknado is a disaster movie indeed! Disasters come in many forms. But none quite like this. It was awful, but funny! It indeed jump the shark."*	Mixed / ironic (mostly negative)	negative	0.1228
2	Interstellar (10/10)	*"Amongst the best movies of all time. The story, the acting, the script, the cinematography, the effects, the sound and the production as a whole is all absolute 10/10's. But what beats all of that is Hans Zimmers compositions. How he continues to churn out perfection to the senses is mindblowing."*	Positive	positive	0.9930
3	Superbad (5/10)	*"A pretty average teen, American comedy. I can respect that it kinda initiated the dumb, adult comedy era that included Ted, let's be cops, 21 jump Street among others, and maybe I would've enjoyed it better at a younger age but honestly not as good as the hype around it."*	Mildly negative / lukewarm	positive	0.8257

Observations.

- The Sharknado review is deliberately ambiguous, it contains "disaster", "awful" and "jump the shark" (all negative signals) alongside "funny" (positive). The model picks up on the negative vocabulary and predicts *negative* at 0.1228. My own reading is that the review is sarcastically *positive* about a bad movie, so the model reaches a defensible but not quite "correct" conclusion, a classic sarcasm failure case for a bag-of-words-ish classifier.
- The Interstellar review is unambiguously positive, and the model is rightly very confident (0.9930). The strong sentiment words ("best", "perfection", "mindblowing") dominate despite the rarer filmmaking vocabulary around them.
- The Superbad review is the most interesting failure: it is lukewarm and leans mildly negative ("not as good as the hype around it"), yet the model predicts *positive* at 0.8257. The review is padded with positive-leaning filler ("I can respect that", "I would've enjoyed it", "comedy") and lists several other comedies by name ("Ted", "21 Jump Street"), which probably nudge the `[CLS]` representation toward the positive side. The mild hedging at the end is not strong enough to flip that back, so the model ends up confidently on the wrong side of the decision boundary.

Overall the results match expectations: solid performance on clearly-worded reviews, and the failure modes are exactly the ones you would expect from a small word-level encoder trained only on IMDb: sarcasm, lukewarm/mixed sentiment, and unusual vocabulary.

Part 2: Decoder-only Model for Text Generation

Decoder Block

The decoder block is implemented similarly to the encoder block described in Part 1, with two sub-layers: a masked multi-head self-attention layer and an MLP block, each wrapped with a residual connection and layer normalization. The key difference from the encoder block is the addition of a causal mask in the attention layer, which prevents each token from attending to future positions, essential for autoregressive text generation. The MLP block follows the same structure as in Part 1, with two linear layers and a GELU activation with a hidden size of 4d.

Model

Embedding Layer

Raw token IDs are integers. They carry no semantic meaning as numbers. The embedding layer is a learned lookup table that maps each token ID to a dense vector of dimension d. These vectors are updated during training, so the model learns to place semantically similar tokens nearby in this d-dimensional space.

Positional Encoding

Attention has no built-in sense of order. It treats its input as a set, not a sequence. Positional encoding fixes this by adding a position-dependent signal to each token embedding before it enters the decoder stack. The standard approach uses fixed sinusoidal functions of different frequencies across the embedding dimensions, so each position gets a unique pattern. This lets the model reason about which token came first, second, etc.

Decoder Block (×N)

Each block has two sub-layers with residual (skip) connections and layer normalization around each:

1. **Masked Multi-Head Self-Attention**

This is where tokens attend to each other. Each token produces a query, key, and value vector. Attention scores are computed between all query-key pairs, softmaxed, and used to weight the values. The result for each token is a context-aware representation that blends information from other tokens. Two masks control which tokens are allowed to attend to which:

Causal mask — enforces left-to-right generation. Token at position i can only attend to positions ≤ i. The mask is an upper-triangular boolean matrix filled with True (meaning "block this") above the diagonal. Without this, the model could cheat during training by looking ahead at the answer it's supposed to predict. Key padding mask — prevents tokens from attending to [PAD] tokens. Padding is added to make all sequences in a batch the same length, but these are meaningless positions. The mask is True at padding positions so they are excluded from attention score computation.

Both masks are applied simultaneously inside MultiheadAttention via the attn_mask and key_padding_mask arguments respectively.

2. **MLP Block (Feed-Forward Network)**

A position-wise two-layer MLP applied independently to each token. It uses a hidden size of 4d and a GELU activation: MLP(x)=Linear(GELU(Linear(x))) This is where most of the model's representational capacity lives. The attention layer routes information between tokens, but the MLP transforms it. Dropout is applied after both sub-layers, and layer normalization is applied before each sub-layer (pre-norm style, as in GPT-2).

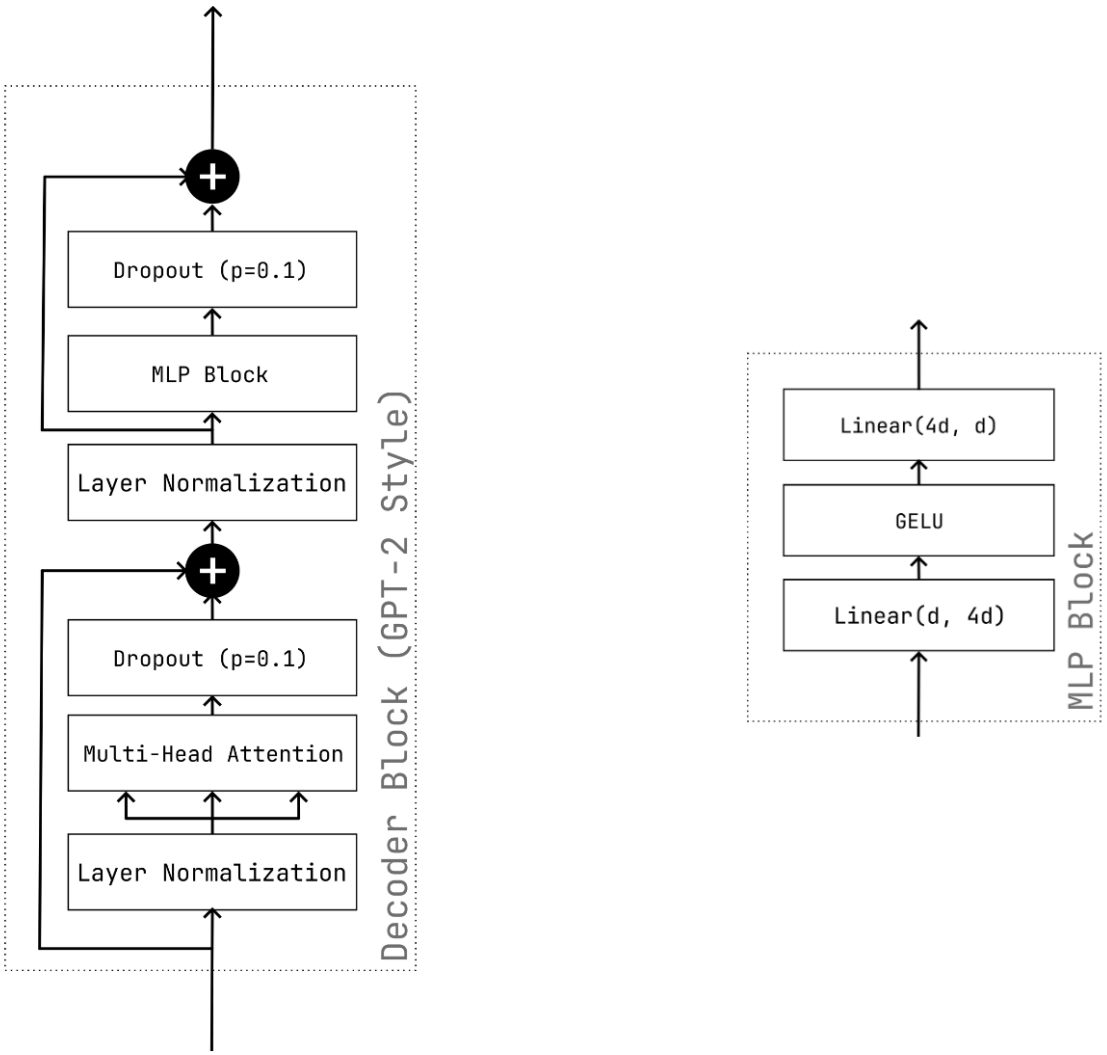

Figure 2: A visual representation of the Decoder Block

Linear Classification Layer

After the N decoder blocks, a single linear layer projects from d dimensions to vocabulary size V. The output at each position is a vector of logits, one per token in the vocabulary representing the unnormalized log-probabilities of what token comes next.

BPE Tokenizer

The BPE tokenizer is trained on the full GooAQ dataset by iteratively merging the most frequent adjacent byte-level character pairs into new subword tokens, until the target vocabulary size of 20,000 tokens is reached. Tokens that appear fewer than 5 times are discarded via the min_frequency threshold. Compared to word-level tokenization, BPE handles rare and out-of-vocabulary words more gracefully. Rather than mapping an unknown word to a single [UNK] token, BPE decomposes it into known subword units. For example, an uncommon word like "unhappiness" might be split into "un", "happiness" rather than replaced wholesale with [UNK]. This results in a more informative representation of rare words and a more compact vocabulary than pure word-level tokenization, which would require a separate token for every unique word in the corpus.

Training

The model was trained for 5 epochs on the full GooAQ subset of 859,765 question-answer pairs, with a batch size of 128 and a learning rate of 0.0001. The model has 36,261,920 parameters with an embedding size of 512, 8 attention heads, and 5 decoder layers. The training loss decreased steadily across all epochs:

Epoch Mean Cross-Entropy Loss 1 4.7823 2 3.8625 3 3.5208 4 3.3187 5 3.1840

The loss decreased consistently across all five epochs with no signs of plateauing, suggesting the model could have benefited from additional training epochs. The largest drop occurred between epoch 1 and 2 (~0.92), with the rate of improvement gradually slowing in later epochs, a typical pattern for cross-entropy loss in language model training. Notably, each epoch took roughly 10–11 minutes on GPU, for a total training time of around 52 minutes. Since no validation set was used, it is difficult to determine whether the model began overfitting at any point. However, given that the training loss was still decreasing at epoch 5 without any sign of flattening, underfitting is more likely the dominant issue, the model simply needed more epochs rather than fewer.

Challenges

The implementation process was largely straightforward. The dataset pipeline, decoder block, and sampling strategies were implemented without major issues, with only minor bugs encountered along the way, such as ensuring the [END] token was preserved after truncation and correctly shifting the source and target sequences by one position. The main challenge was the quality of the model outputs.

Prediction analysis

The model's predictions were generally poor. Generated answers rarely had any meaningful connection to the input question and often consisted of incoherent or repetitive text. This is likely a consequence of the model being relatively small (~36M parameters) and trained on a limited subset of the dataset for five epochs, which is insufficient for a language model of this complexity to generalize well.

Sampling strategy. Greedy sampling tended to produce repetitive, deterministic outputs, often getting stuck in loops where the same token or phrase was repeated until the maximum sequence length was reached. This is a well-known failure mode of greedy decoding. Top-p sampling produced more varied outputs due to the stochastic sampling from the nucleus, but the added randomness also introduced more incoherence. At higher temperatures the outputs degraded into near-random token sequences, while lower temperatures brought the behavior closer to greedy.

Figure 3: Here we can se the results from trying to converse with our model. The results are clearly incorect and often make no sense at all.

Overfitting. Given that training loss decreased quickly to around 3 but the model produced poor answers at inference time, there are signs of overfitting to surface-level token patterns rather than learning to model question-answer relationships. The dataset implementation in QADataset truncates long sequences and forces an [END] token at position max_length, which means many answers are cut off mid-sentence during training. This likely hurt the model's ability to learn coherent answer structure.

Hallucinations. The model frequently generated fluent-sounding but factually wrong or nonsensical answers, which is a classic hallucination pattern. Since the model has no retrieval mechanism and limited training, it has no reliable grounding in facts. It simply learns to produce token sequences that statistically resemble answers in the training data.

An example of these problems is when we asked the model what Bergen is and it replied "dova is a form of abnormar (alkali) nateur (alkali) (nate) with blood samples". This is clearly not what bergen is, and tells us that out model is not good enough to generalize to new questions.

How would we improve the model?

Given more time and resources, the most impactful improvement would be to replace the from-scratch training approach with fine-tuning a pretrained model such as GPT-2 or a similar publicly available language model. These models have already been trained on massive text corpora and have learned rich representations of language, meaning fine-tuning on the QA dataset would require far less data and compute to achieve coherent outputs.

Beyond that, several other improvements would likely help:

More data and longer training. The model was trained on a subset of the dataset for a few epochs. Training on the full dataset for multiple epochs with a learning rate schedule (such as cosine annealing with warmup) would give the model a much better chance of learning meaningful question-answer patterns.

Larger model. Increasing the number of decoder blocks, the embedding dimension, or the number of attention heads would give the model more representational capacity, at the cost of compute.

Better sampling. Implementing more sophisticated decoding strategies such as beam search or top-k combined with top-p sampling could improve output quality without any changes to the model itself.

Validation set and early stopping. The current setup has no validation set, making it difficult to detect overfitting during training. Adding one and using early stopping would help select the best checkpoint rather than simply the last one.

Longer sequence length. Many answers were truncated during training due to the max_length constraint, which likely hurt the model's ability to learn complete answer structure. More compute would allow a larger max_length.

Disclosure of AI

The service ChatGPT and Claude has been used for language editing and or improvements to the text in the report, and for resolving bugs. Claude was also used to assist in creating the MultiheadAttention class. The final result was fact-checked and somewhat rewritten by its authors.